

INTERNSPARK INTERNSHIP

RTL Design & Functional Simulation (Verilog)

Task 1 Report

Name	Devunoori Harshitha
College	Narsimha Reddy Engineering College
Roll Number	24X01A0419
Task	Task 1 - RTL Design & Functional Simulation
Date	26 May 2026

1. Goal

Design an 8-bit ALU (Arithmetic Logic Unit) in Verilog and verify its functionality using a testbench with directed test cases.

2. RTL Design - 8-bit ALU Module

The ALU supports 4 operations: ADD, SUB, AND, OR.

```
module alu_8bit(  
    input [7:0] a, b,  
    input [2:0] op,  
    output reg [7:0] result  
);  
always @(*) begin  
    case(op)  
        3'b000: result = a + b; // ADD  
        3'b001: result = a - b; // SUB  
        3'b010: result = a & b; // AND  
        3'b011: result = a | b; // OR  
        default: result = 8'b0;  
    endcase  
end  
endmodule
```

3. Testbench

```
module tb_alu;  
    reg [7:0] a, b;  
    reg [2:0] op;  
    wire [7:0] result;  
    alu_8bit uut(.a(a), .b(b), .op(op), .result(result));  
    initial begin  
        $dumpfile("dump.vcd");  
        $dumpvars(0, tb_alu);  
        a=10; b=5; op=3'b000; #10;  
        a=10; b=5; op=3'b001; #10;  
        a=10; b=5; op=3'b010; #10;  
        a=10; b=5; op=3'b011; #10;  
        $finish;  
    end  
endmodule
```

Code Screenshot



The image shows two side-by-side windows from a Verilog IDE. The left window, titled 'testbench.vv', contains a testbench module 'tb_alu' that instantiates an 'alu_8bit' module and performs several operations with specific values and delays. The right window, titled 'design.vv', contains the 'alu_8bit' module definition, which implements an 8-bit ALU with inputs 'a' and 'b' (7-bit), an operation input 'op' (2-bit), and an 8-bit output 'result'. The module uses a case statement to handle different operations: addition, subtraction, AND, and OR.

```
1 module tb_alu;
2   reg [7:0] a, b;
3   reg [2:0] op;
4   wire [7:0] result;
5   alu_8bit uut(.a(a),.b(b),.op(op),.result(result));
6   initial begin
7     $dumpfile("dump.vcd");
8     $dumpvars(0, tb_alu);
9     a=10; b=5; op=3'b000; #10;
10    a=10; b=5; op=3'b001; #10;
11    a=10; b=5; op=3'b010; #10;
12    a=10; b=5; op=3'b011; #10;
13    $finish;
14  end
15 endmodule
16
```

```
1 module alu_8bit(
2   input [7:0] a, b,
3   input [2:0] op,
4   output reg [7:0] result
5 );
6 always @(*) begin
7   case(op)
8     3'b000: result = a + b;
9     3'b001: result = a - b;
10    3'b010: result = a & b;
11    3'b011: result = a | b;
12    default: result = 8'b0;
13  endcase
14 end
15 endmodule
16
```

4. Simulation Output

Simulated using JDoodle Online Verilog Compiler. All operations verified correct.

Operation	A	B	Result
ADD (10+5)	10	5	15
SUB (10-5)	10	5	5
AND (10&5)	10	5	0
OR (10 5)	10	5	15

Output Screenshot

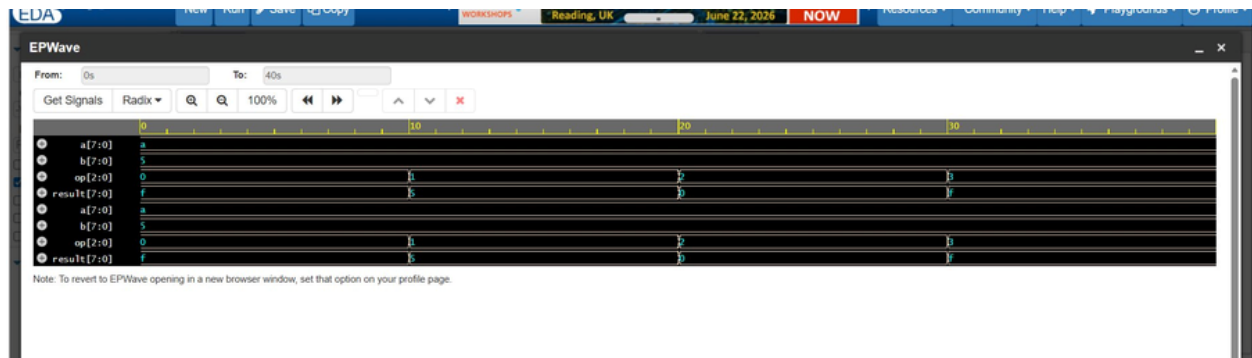
Output Generated files

```
ADD: 15
SUB: 5
AND: 0
OR: 15
jdoodle.v:31: $finish called at 40 (1s)
|
```

 Compiled and executed in 0.817 sec(s)

5. Simulation Waveform

Waveform generated using EDA Playground (EPWave) with Icarus Verilog 12.0. Shows correct signal transitions for all 4 operations.



6. Coverage Summary

All 4 ALU operations tested with directed test cases (a=10, b=5):

- ADD: $10 + 5 = 15$ ✓
- SUB: $10 - 5 = 5$ ✓
- AND: $10 \& 5 = 0$ ✓
- OR: $10 | 5 = 15$ ✓

Tool: EDA Playground with Icarus Verilog 12.0 + EPWave for waveform visualization.